

IMAGE PROCESSING WITH MATRICES

Software Requirements Specification (SRS)

Trainee Implementation Guide

Version 1.0 | C++ Data Structures Training Module

Prepared for: Software Engineering Trainees

Language: C++ (2D/3D Vectors, Image Matrices & Pixel Operations)

INTERNAL TRAINING DOCUMENT — NOT FOR PUBLIC DISTRIBUTION

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document defines the complete functional requirements for implementing seven image-processing operations in C++. It is intended to guide trainees through completing a partially-written codebase by specifying exactly what each function must do.

By the end of this exercise, trainees will demonstrate mastery of:

- Representing images as 3-dimensional vectors in C++
- Iterating over pixel grids with nested loops
- Applying mathematical formulas to pixel values (grayscale, brightness, contrast)
- Implementing spatial transformations (flip, rotate)
- Applying convolution-style neighbourhood filters (blur)
- Clamping values to the valid [0, 255] range
- Writing clean, correct nested-loop code without off-by-one errors

1.2 Project Overview

The Image class stores pixel data as a 3D vector indexed by [height][width][channel]. The skeleton code — the full Image class with load/save/print support plus an empty body for each processing function and a complete test harness in main() — is provided. Trainees must implement all function bodies marked // TODO.

1.3 Scope

The implementation covers the following seven operations:

- convertToGrayscale — luminance-weighted conversion from RGB to single-channel grey
- flipHorizontal — mirror image left-to-right
- flipVertical — mirror image top-to-bottom
- adjustBrightness — add a constant offset to every channel
- adjustContrast — scale channel values around the midpoint 128
- applyBlur — 3x3 averaging filter over all interior pixels
- rotate90 — 90-degree clockwise rotation with transposed dimensions

1.4 Definitions and Acronyms

Term	Definition
Pixel	The smallest unit of an image, holding one value per channel (R, G, B or grey).
Channel	A single colour component of a pixel. RGB images have 3 channels (0=R, 1=G, 2=B). Grayscale images have 1.
data[y][x][c]	The pixel at row y, column x, channel c inside the Image class.
Clamp	Restrict a value to the range [0, 255]: $\max(0, \min(255, \text{value}))$.
Luminance	The perceived brightness of a colour, computed as $0.299 \cdot R + 0.587 \cdot G + 0.114 \cdot B$.
Kernel / Filter	A small matrix (here 3x3) used to compute a weighted neighbourhood sum.

PPM (P3)	A plain-text image format. Each pixel is stored as three integers (R G B).
Trainee	A junior developer completing this exercise.

2. Data Structure Design

2.1 The Image Class

The Image class is already fully implemented. Do not modify it. It stores pixel data in:

Field	Description
width, height	Dimensions of the image in pixels.
channels	Number of colour channels per pixel (1 for grayscale, 3 for RGB).
data[height][width][channels]	3D vector of integers. Access a pixel with data[y][x][c] or operator()(y, x, c).

2.2 Pixel Access

Throughout the implementation use the overloaded operator to read and write pixels:

```
input(y, x, c)    // read pixel at row y, column x, channel c
output(y, x, c)   // write pixel
```

2.3 Index Conventions

Rows are indexed 0 to height-1 (top to bottom). Columns are indexed 0 to width-1 (left to right). Channels: 0 = Red, 1 = Green, 2 = Blue for colour images; 0 = Grey for grayscale.

3. Function Specifications

3.1 convertToGrayscale(const Image& input)

Converts a 3-channel RGB image to a single-channel grayscale image using the standard luminance formula.

Return type: Image (single-channel, same width and height as input)

Algorithm — implement these steps exactly:

- Create output image: Image output(width, height, 1)
- For each row y from 0 to height-1:
 - For each column x from 0 to width-1:
 - Read R = input(y, x, 0), G = input(y, x, 1), B = input(y, x, 2)
 - Compute gray = (int)(0.299*R + 0.587*G + 0.114*B)
 - Set output(y, x, 0) = gray
- Return output

Formula:

```
gray = (int) (0.299 * R + 0.587 * G + 0.114 * B)
```

Example — Red pixel (255, 0, 0):

```
gray = (int) (0.299*255 + 0.587*0 + 0.114*0) = (int) (76.245) = 76
```

Note: No clamping is needed here because input values are already in [0,255] and the weights sum to 1.0.

3.2 flipHorizontal(const Image& input)

Mirrors the image left-to-right. The pixel at column x in the input appears at column $(width-1-x)$ in the output.

Return type: Image (same dimensions and channels as input)

Algorithm:

- Create output with the same width, height, and channels
- For each y, x, c :
 - $output(y, width-1-x, c) = input(y, x, c)$
- Return output

Visual example (4-pixel wide row):

```
Input  col:  0   1   2   3
         A   B   C   D
Output col:  0   1   2   3
         D   C   B   A
```

3.3 flipVertical(const Image& input)

Mirrors the image top-to-bottom. The pixel at row y in the input appears at row $(height-1-y)$ in the output.

Return type: Image (same dimensions and channels as input)

Algorithm:

- Create output with the same width, height, and channels
- For each y, x, c :
 - $output(height-1-y, x, c) = input(y, x, c)$
- Return output

3.4 adjustBrightness(const Image& input, int value)

Adds a constant integer offset to every channel of every pixel, then clamps the result to $[0, 255]$. Positive values brighten the image; negative values darken it.

Return type: Image (same dimensions and channels as input)

Algorithm:

- Create output with the same width, height, and channels
- For each y, x, c :
 - $new_val = input(y, x, c) + value$
 - $output(y, x, c) = \max(0, \min(255, new_val))$
- Return output

Example (value = +50):

```
Input pixel (200, 30, 10) -> Output pixel (250, 80, 60)
Input pixel (220, 10, 5)  -> Output pixel (255, 60, 55) <- clamped at 255
```

3.5 adjustContrast(const Image& input, float factor)

Scales the contrast of the image by stretching or compressing pixel values around the midpoint 128. A factor > 1 increases contrast; $0 < factor < 1$ reduces it.

Return type: Image (same dimensions and channels as input)

Formula:

```
new_val = factor * (input(y, x, c) - 128) + 128
output(y, x, c) = max(0, min(255, (int)new_val))
```

Algorithm:

- Create output with the same width, height, and channels
- For each y, x, c:
 - $\text{new_val} = \text{factor} * (\text{input}(y, x, c) - 128) + 128$
 - $\text{output}(y, x, c) = \max(0, \min(255, (\text{int})\text{new_val}))$
- Return output

Example (factor = 1.5):

```
Pixel 200: 1.5*(200-128)+128 = 1.5*72 +128 = 236
Pixel 50: 1.5*(50 -128)+128 = 1.5*(-78)+128 = 11
Pixel 240: 1.5*(240-128)+128 = 1.5*112+128 = 296 -> clamped to 255
```

3.6 applyBlur(const Image& input)

Applies a 3x3 box blur. Each output pixel is the arithmetic mean of the 9 pixels in its 3x3 neighbourhood. Border pixels (row 0, row height-1, col 0, col width-1) are left as 0.

Return type: Image (same dimensions and channels as input)

Algorithm:

- Create output with the same width, height, and channels (initialised to 0)
- For each interior pixel y from 1 to height-2, x from 1 to width-2:
 - For each channel c:
 - $\text{sum} = 0$
 - For k_y in $\{-1, 0, 1\}$:
 - For k_x in $\{-1, 0, 1\}$:
 - $\text{sum} += \text{input}(y+k_y, x+k_x, c)$
 - $\text{output}(y, x, c) = \text{sum} / 9$
 - Return output

3x3 neighbourhood diagram:

```
(y-1, x-1) (y-1, x) (y-1, x+1)
(y, x-1) (y, x) (y, x+1)    <- centre pixel being computed
(y+1, x-1) (y+1, x) (y+1, x+1)
```

Note: Integer division ($\text{sum} / 9$) is intentional — fractional parts are discarded.

3.7 rotate90(const Image& input)

Rotates the image 90 degrees clockwise. The output dimensions are swapped: output width = input height, output height = input width.

Return type: Image (dimensions transposed)

Algorithm:

- Create output as Image(height, width, channels) <- note swapped arguments
- For each y from 0 to input_height-1:
 - For each x from 0 to input_width-1:
 - For each channel c:
 - $\text{output}(x, \text{height}-1-y, c) = \text{input}(y, x, c)$
 - Return output

Mapping rule:

```
output row    = x          (column in input becomes row in output)
output column = height-1-y (flipped row in input becomes column)
```

Example — 2x3 input becomes 3x2 output:

```
Input (2 rows, 3 cols):      Output (3 rows, 2 cols):
A B C                        G D A
D E F                        H E B
                              I F C
```

4. Edge Cases

Your implementation must correctly handle all of the following situations:

Situation	Expected Behaviour	Why It Matters
Brightness pushes channel above 255	Clamp to 255	Pixel values must stay in [0,255] for valid PPM output
Brightness pushes channel below 0	Clamp to 0	Negative pixel values are invalid
Contrast factor = 1.0	Output identical to input	Multiplying by 1 around 128 returns the original value
Contrast factor = 0.0	All pixels set to 128	Everything maps to the midpoint
applyBlur on 1x1 or 2x2 image	Output is all zeros (no interior pixels)	The loop <code>y=1..height-2</code> never executes
rotate90 on a non-square image	Width and height are swapped in output	A 4x2 image becomes a 2x4 image
Grayscale of pure black (0,0,0)	Output grey = 0	Arithmetic gives 0
Grayscale of pure white (255,255,255)	Output grey = 255	Weights sum to 1, so result equals input

5. Method Summary

Function	Signature	Purpose
convertToGrayscale	Image f(const Image&)	Luminance-weighted RGB to grey
flipHorizontal	Image f(const Image&)	Mirror left <-> right
flipVertical	Image f(const Image&)	Mirror top <-> bottom
adjustBrightness	Image f(const Image&, int)	Add constant offset, clamp
adjustContrast	Image f(const Image&, float)	Scale around midpoint 128, clamp
applyBlur	Image f(const Image&)	3x3 averaging filter on interior pixels
rotate90	Image f(const Image&)	90-degree clockwise rotation, swap dimensions

6. Expected Test Output

Compile and run with:

```
g++ -std=c++17 -Wall -Wextra image_processing.cpp -o img && ./img
```

6.1 Grayscale

The test image contains known colours. After `convertToGrayscale` the pixel at (0,0) — originally Red (255,0,0) — should be:

```
R=255, G=0, B=0 -> gray = (int) (0.299*255) = 76
```

6.2 Horizontal Flip

Row 0 of the flipped image must be [White, Blue, Green, Red] — the reverse of the original [Red, Green, Blue, White].

6.3 Vertical Flip

Row 0 of the vertically flipped image must equal Row 3 of the original. The first pixel of row 0 after flipping should be (128, 255, 128) — Light Green.

6.4 Brightness +50

The Red pixel (255, 0, 0) becomes (255, 50, 50). The Gray pixel (128, 128, 128) becomes (178, 178, 178).

6.5 Contrast x1.5

The Red pixel (255, 0, 0): $R \rightarrow 1.5 \cdot (255 - 128) + 128 = 318.5 \rightarrow 255$ (clamped). $G \rightarrow 1.5 \cdot (0 - 128) + 128 = -64 \rightarrow 0$ (clamped). Result: (255, 0, 0).

6.6 Blur

Border pixels remain 0. Interior pixel (1,1) in the Red channel is the average of the 9 pixels in its 3x3 neighbourhood. Verify the arithmetic matches the printed grid.

6.7 Rotate 90 Degrees

The 4x4 image remains 4x4. The pixel that was at (0,0) (Red) now appears at output position (0, 3). The pixel at (0,3) (White) now appears at (3, 3).

7. Common Mistakes & Tips

Mistake	Why It's Wrong	Correct Approach
Forgetting to clamp in brightness or contrast	Values outside [0,255] corrupt the PPM file	Always apply $\max(0, \min(255, \text{value}))$ before writing to output

Integer arithmetic in contrast without casting	Truncation gives wrong results when multiplying by a float	Compute as float first, then cast: <code>output = (int)(factor*(v-128)+128)</code>
Writing to input directly instead of output	Modifies source data mid-loop, producing incorrect results	Always create a separate output Image and write only to it
Blurring border pixels	Reads out-of-bounds neighbours — undefined behaviour	Loop from <code>y=1</code> to <code>height-2</code> and <code>x=1</code> to <code>width-2</code>
Swapping arguments in <code>rotate90</code> output constructor	Output has wrong dimensions	Rotated output must be <code>Image(height, width, channels)</code>
Grayscale: not casting to int before storing	Floating-point value truncates silently	Always cast: <code>output(y,x,0) = (int)(0.299*R + 0.587*G + 0.114*B)</code>
Missing channel loop in flips / brightness	Only channel 0 is copied; others stay 0	Nest a <code>for(int c=0; c<channels; c++)</code> loop inside the <code>y/x</code> loops

8. Implementation Checklist

Use this checklist to track your progress. Every item must be completed before submission.

8.1 Grayscale

#	Item	Done?
1	Output image created with 1 channel	☐
2	Correct luminance formula applied: $0.299R + 0.587G + 0.114B$	☐
3	Result cast to int before storing in output	☐

8.2 Flips

#	Item	Done?
4	<code>flipHorizontal</code> : output column = <code>width-1-x</code>	☐
5	<code>flipVertical</code> : output row = <code>height-1-y</code>	☐
6	Both flips iterate over all channels	☐

8.3 Brightness & Contrast

#	Item	Done?
7	<code>adjustBrightness</code> : adds value to each channel	☐
8	<code>adjustBrightness</code> : clamps result to <code>[0, 255]</code>	☐
9	<code>adjustContrast</code> : applies <code>factor*(v-128)+128</code> formula	☐
10	<code>adjustContrast</code> : clamps result to <code>[0, 255]</code>	☐

8.4 Blur

#	Item	Done?
11	Only interior pixels (y: 1..height-2, x: 1..width-2) are processed	[]
12	3x3 neighbourhood sum computed with nested ky/kx loops	[]
13	Sum divided by 9 (integer division)	[]

8.5 Rotate

#	Item	Done?
14	Output created as Image(height, width, channels) — dimensions swapped	[]
15	Mapping: output(x, height-1-y, c) = input(y, x, c)	[]

9. Grading Criteria

Criterion	Weight / Description
Correctness of Output	40% — All 7 transformations produce pixel values matching the expected results in Section 6
Edge Case Handling	25% — Clamping, blur borders, non-square rotate, pure black/white grayscale all handled correctly
Loop Structure	20% — Correct nesting of y/x/channel loops; no off-by-one errors; blur uses proper ky/kx offsets
Code Quality	10% — Readable code, consistent variable naming, no unnecessary copies or dead code
Compilation	5% — Compiles without warnings: g++ -std=c++17 -Wall -Wextra

Appendix — Quick Reference Card

A concise summary for quick lookup during implementation.

Function Signatures and Return Values

Function	Signature	Returns
convertToGrayscale	Image f(const Image& input)	Single-channel Image
flipHorizontal	Image f(const Image& input)	Same-size Image, mirrored horizontally
flipVertical	Image f(const Image& input)	Same-size Image, mirrored vertically
adjustBrightness	Image f(const Image& input, int value)	Same-size Image, shifted & clamped

adjustContrast	Image f(const Image& input, float factor)	Same-size Image, scaled & clamped
applyBlur	Image f(const Image& input)	Same-size Image, blurred interior
rotate90	Image f(const Image& input)	Image with swapped dimensions

Key Formulas at a Glance

```

Grayscale:  gray = (int)(0.299*R + 0.587*G + 0.114*B)
Clamp:      v    = max(0, min(255, v))
Brightness: out  = clamp(input(y,x,c) + value)
Contrast:   out  = clamp((int)(factor * (input(y,x,c) - 128) + 128))
Blur:       out(y,x,c) = sum_of_9_neighbours / 9
FlipH:      out(y, width-1-x, c) = in(y, x, c)
FlipV:      out(height-1-y, x, c) = in(y, x, c)
Rotate90:   out(x, height-1-y, c) = in(y, x, c)    [output = Image(height,width,ch)]

```

End of Document